Backtracking Review Sheet

Backtracking is one recursive skeleton reused across a whole family of "find every possibility" problems: subsets, permutations, combination sums, valid parentheses, letter combinations, word search. Every variant is the same three moves — **choose, recurse deeper, undo the choice** — with the details of what "choose" and "when to stop" mean changing from problem to problem.

Python 3 · Tutor style · Includes diagrams · Includes FAQ

00 The Universal Template

Every problem in this book is a variation on the same three-line skeleton. Learn to see it underneath the surface differences.

Universal backtracking skeleton — template

```
def backtrack(...):
    if :
        result.append(path.copy())   # save a snapshot (must copy!)
        return

    for choice in :
        # make the choice
        path.append(choice)
        backtrack(...)               # recurse deeper
        # undo the choice
        path.pop()
```


02 ← start — Subsets — choose / skip

03 — Permutations — used[] array

04 — Combination Sum — reuse allowed

05 — No-Reuse — index+1

06 — Parentheses — pruning

07 — Letter Combos — mapping

08 — Word Search — grid DFS

> **The order never changes:** append → backtrack → pop. If you write append → pop → backtrack, you've already undone the choice before you've explored what it leads to — the next level down never even sees it. The sequence's meaning is: **carry the choice deeper, then walk it back once you're done exploring.**

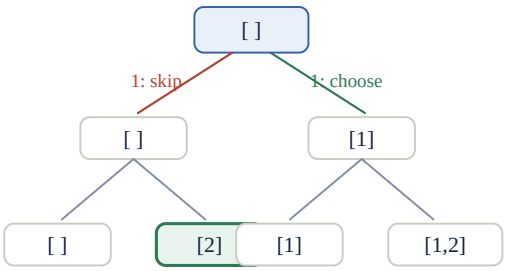
01 The Backtracking Concept

Backtracking is systematic trial-and-error: try a choice, explore everything that follows from it, then undo it and try the next choice — visiting every possibility in the search space exactly once.

> **Three questions to ask for every new problem:** ① What does `path` represent at any moment (the choices made so far)? ② What's the **stopping condition** — when is a path complete and worth recording? ③ At each level, what are the **legal choices**, and does a choice get removed from future consideration (permutations, no-reuse combinations) or stay available (subsets, reusable combinations)?

02 Subsets · choose / skip

For every number, branch into two choices: include it, or don't. `[1,2,3]` and `[2,3]` are two different subsets.



... level 3 decides on 3, leaves are all 8 subsets

One number per level; left branch = skip, right branch = choose — every path to the bottom is one subset

subsets — expand choose/skip at every level

subsets

```python
def subsets(numbers):
    result = []
    path = []

    def backtrack(index):
        if index == len(numbers):     # every number has been decided
            result.append(path.copy())  # snapshot! not path itself
            return

        # skip the current number
        backtrack(index + 1)

        # choose the current number
        path.append(numbers[index])
        backtrack(index + 1)
        path.pop()                    # undo, back to the "skip" world

    backtrack(0)
    return result
```

`index: which number we're deciding on now` `path: the choices made along the current route`

`result: every completed subset`

Q Why must you use `path.copy()`, not just `result.append(path)`?

Because `path` keeps being modified by later `append()` / `pop()` calls after this point. Storing `path` directly stores a **reference** — by the time backtracking finishes, every element in `result` would point to the *same* (now emptied) list. `path.copy()` takes a snapshot of this exact moment, which is the subset you actually meant to save.

Q Why can't the order of append / pop / recurse be shuffled?

It must be `append → backtrack → pop`. Write `append → pop → backtrack` instead, and you've undone the choice before descending a level — the level below never even knows you made it. The order's meaning is: **carry the choice deeper, then walk it back once you're done exploring.**

03 Permutations · must master

Every number **must** be used exactly once, but order matters — `[1,2]` and `[2,1]` are two different answers. Every level re-scans the entire number list looking for "whichever haven't been used yet."

permutations — used[] tracks what's already placed used

```python
def permutations(numbers):
    result = []
    path = []
    used = [False] * len(numbers)

    def backtrack():
        if len(path) == len(numbers):   # placed all of them → done
            result.append(path.copy())
            return

        for index in range(len(numbers)):  # every level scans everyone again
            if used[index]:
                continue                   # already placed, skip
            path.append(numbers[index])
            used[index] = True
            backtrack()
            path.pop()                      # undo: both must roll back!
            used[index] = False

    backtrack()
    return result
```

Q Why doesn't this call `backtrack(index + 1)` like Subsets does?

Subsets' `for` loop only walks the number list once, forward, so it can advance with `index + 1`. Permutations needs to **re-scan all positions from scratch at every level**, because any not-yet-used number could go next — there's no single forward-moving index that captures that, so it needs the `used` array instead.

Q Why must `path.pop()` and `used[index] = False` both appear?

They undo two different pieces of state: `path.pop()` removes the number from the current route; `used[index] = False` makes that number available again for other branches. Skip either one and the two pieces of state fall out of sync — you'll either leak numbers into paths that shouldn't have them, or permanently lock a number out.

04 Combination Sum · reuse allowed

Find every combination of candidates summing to `target` — the same number can be reused unlimited times. `combination_sum([2,3,6,7], 7)` → `[[2,2,3], [7]]`.

combination_sum — remaining + start_index, reuse via same index reuse

```python
def combination_sum(candidates, target):
    result = []
    path = []

    def backtrack(start_index, remaining):
        if remaining == 0:              # hit target exactly → valid combination
            result.append(path.copy())
            return
        if remaining < 0:               # overshot → dead end, back out
            return

        for index in range(start_index, len(candidates)):
            path.append(candidates[index])
            backtrack(index, remaining - candidates[index])  # pass index, not index+1!
            path.pop()

    backtrack(0, target)
    return result
```

Trace for target=7: choose 2 → remaining 5; choose 2 again → remaining 3; choose 3 → remaining 0 ✓ — result `[2,2,3]` . Two things need explaining: `remaining` tracks "how much is still left to make," updated by `remaining - candidates[index]` and checked against `== 0` / `< 0` . `start_index` keeps the loop from restarting at 0 every level — without it, `[2,2,3]` and `[2,3,2]` would both get generated as if they were different answers, when they're really the same combination in a different order.

Q Why pass `index` (not `index + 1`) into the recursive call here?

Because the same number is allowed to be reused. Passing `index` keeps the current candidate available to be chosen again on the next level down; passing `index + 1` would move past it, permanently ruling out reuse (that's exactly what § 05's no-reuse variant does instead).

05 The No-Reuse Variant: `index` → `index + 1`

This whole family of problems (Combination Sum II, Subsets II, etc.) hinges on a single one-character difference from § 04.

Reuse allowed vs. not — the only line that changes — index vs index+1

```
# Reuse allowed:
backtrack(index, remaining - candidates[index])

# Each item used at most once ("no reuse"):
backtrack(index + 1, remaining - candidates[index])
```

The rule of thumb: if a candidate can be picked again after being picked, the next call passes `index` (same position stays reachable). If each candidate can be picked at most once, the next call passes `index + 1` (advance past it, so it's off the table for the rest of this branch). Everything else about the template — the `for start_index in range(...)` loop, the `remaining == 0` / `< 0` checks, append/backtrack/pop — stays identical.

06 Generate Parentheses · pruning

Given `n` pairs of parentheses, generate every valid combination. `generate_parentheses(2)` → `["(())", "()()"]`.

generate_parentheses — prune illegal branches before they're built — prune

```
def generate_parentheses(n):
    result = []
    path = []

    def backtrack(open_count, close_count):
        if len(path) == 2 * n:           # used up all n pairs
            result.append("".join(path))
            return

        if open_count < n:               # still have '(' left to place
            path.append("(")
            backtrack(open_count + 1, close_count)
            path.pop()

        if close_count < open_count:       # a ')' would still be legal here!
            path.append(")")
            backtrack(open_count, close_count + 1)
            path.pop()

    backtrack(0, 0)
    return result
```

The two guard conditions ARE the whole algorithm: `open_count < n` stops you from placing more open parens than you have; `close_count < open_count` stops you from ever placing a close paren before its matching open exists (which would produce something like `")("`). Instead of generating every string of length `2n` and filtering the valid ones afterward, these two checks **prune** illegal branches before they're ever built — the search tree only ever contains legal partial strings.

Q Why `"".join(path)` instead of `path.copy()`?

Subsets and Permutations save a *list* of chosen items, so they need `.copy()` to snapshot it. Here the target output is a single **string** like `"(())"`, and `"".join(path)` both converts the character list into that string *and* creates a brand-new independent object — you get the copy for free as a side effect of the join.

07 Letter Combinations of a Phone Number

2→abc, 3→def, etc. Given digits `"23"`, produce every possible letter combination: `["ad","ae","af","bd","be","bf","cd","ce","cf"]`.

Look up the choice set through a mapping — mapping

```
phone = {"2": "abc", "3": "def", "4": "ghi", ...}

def backtrack(index):
    if index == len(digits):
        result.append("".join(path))
        return
    for letter in phone[digits[index]]:  # this digit's set of letters
        path.append(letter)
        backtrack(index + 1)             # move to the next digit
        path.pop()
```

The pattern is identical to Subsets and Permutations — the only thing that changed is *where the choice set comes from*. Instead of "every remaining number" or "every unused number," the choices at each level are "whichever letters this specific digit maps to." Once you see backtracking as "pick from whatever the current level's legal choice set is," each new problem just becomes: what's my path, what's my stopping condition, and what's my choice set?

08 Word Search — Backtracking on a Grid

Combines DFS on a 2D grid (Book 05's Graph techniques) with backtracking's add/explore/remove rhythm: `visited` plays the same role `path` played earlier.

dfs — grid backtracking — grid DFS

```
def dfs(row, col, index):
    if index == len(word):
        return True                      # matched every character
    if row < 0 or row >= rows or col < 0 or col >= cols:
        return False                      # out of bounds
    if (row, col) in visited:
        return False                      # already used in this path
    if grid[row][col] != word[index]:
        return False                      # letter doesn't match

    visited.add((row, col))                # choose: mark this cell as used
    found = (dfs(row + 1, col, index + 1)  # explore in all 4 directions
             or dfs(row - 1, col, index + 1)
             or dfs(row, col + 1, index + 1)
             or dfs(row, col - 1, index + 1))
    visited.remove((row, col))              # undo! this IS the pop() for a set

    return found
```

Q Why does the function end with a separate `return found` instead of returning directly from inside?

Because `visited.remove((row, col))` — the undo step — has to run regardless of whether this cell's search succeeded or failed, and it has to run **after** the four recursive calls finish but **before** the function actually returns. If you tried to `return dfs(...) or dfs(...) or ...` directly, Python's short-circuit evaluation means `or` stops calling functions the moment one returns True, and the function would exit before the cleanup line ever runs — leaving `visited` permanently polluted for future calls.

Q Why is `visited.remove((row, col))` the same idea as `path.pop()`?

Both undo exactly one choice made on the way in, so that sibling branches explore a clean state. `path` is a list, so its undo operation is `.pop()`; `visited` is a set, so its undo operation is `.remove(...)` — different containers, same backtracking rhythm: **choose → explore → undo**.

09 Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
`result.append(path)`	Stores a reference; every saved answer ends up pointing to the same mutated list	`result.append(path.copy())`
`append → pop → backtrack`	Undoes the choice before descending — the next level never sees it	`append → backtrack → pop`
Permutations without `used[]`	No way to know which numbers are already placed when re-scanning every level	Track with a `used` array; unset it during undo
`backtrack(index, remaining)` forgets to subtract	remaining never shrinks, target is never reached	`remaining - candidates[index]`
No-reuse problem uses `index` instead of `index+1`	Same candidate can be picked again — silently allows reuse	Pass `index + 1`
`return dfs(...) or dfs(...) or ...` directly	Short-circuit `or` can exit before the visited.remove cleanup line runs	Store in a variable, clean up, `return found`
Word Search forgets `visited.remove((r,c))`	Cell stays permanently marked used, blocking valid future paths	Always undo on the way back up, success or failure
Generate Parentheses without pruning	Builds every length-2n string then filters — wasteful and easy to get wrong	Guard with `open_count < n` / `close_count < open_count`

10 Exercises + Checklist

◆ EXERCISES

1. `Easy` Trace `subsets([1,2])` by hand — draw the choose/skip tree and list all 4 resulting subsets.
2. `Medium` Trace `permutations([1,2,3])` — verify you get all 6 orderings, and explain why `used` is necessary here but not in Subsets.
3. `Medium` `combination_sum([2,3,5], 8)` — hand-compute the valid combinations, paying attention to which index gets reused.
4. `Medium` Convert § 04's reuse-allowed version into a no-reuse version (each candidate used at most once) — identify the single line that changes.
5. `Easy` `generate_parentheses(3)` — list all 5 valid combinations, and explain why `close_count < open_count` prevents an invalid `")("` from ever appearing.

Chapter checklist

- [ ] Every variant is append → backtrack → pop
- [ ] Save with .copy() or a fresh object, never a live reference
- [ ] Subsets: choose/skip at every level
- [ ] Permutations: used[] tracks placed numbers
- [ ] Combination Sum: index=reuse, index+1=no reuse
- [ ] remaining==0 accept, remaining<0 reject
- [ ] Parentheses: prune with open/close counts, not filter after
- [ ] Word Search: visited.remove() undoes exactly like path.pop()

> **Once every box is checked,** you should be able to look at any "find all X" problem and immediately identify its path, its choice set, and its stopping condition — the code follows automatically from there.